

Proyecto de Ingeniería de Software - Plantilla Integrada CMMI

Departamento de Ciencias de la Computación - ESPE

Título del Proyecto: SmartRail

Integrantes / Autores:

Nombre	José Yáñez
Rol	Product Owner
Categoría profesional	Programador de desarrollo
Responsabilidad	Verificar y controlar el encaminamiento del trabajo
Información de contacto	jgyanez2@espe.edu.ec

Nombre	Mateo Unda
Rol	Scrum Master
Categoría profesional	Programador de desarrollo
Responsabilidad	Analizar los requerimientos del software, liderar al equipo de trabajo en Scrum
Información de contacto	mjunda1@espe.edu.ec

Nombre	Daniela Narváez
Rol	Scrum Team
Categoría profesional	Programador de desarrollo
Responsabilidad	Diseñar y programar el producto de software
Información de contacto	dmnarvaez6@espe.edu.ec

Nombre	Anthony Vilaña
Rol	Scrum Team
Categoría profesional	Programador de desarrollo
Responsabilidad	Diseñar y programar el producto de software
Información de contacto	aavilana@espe.edu.ec

Fecha: 12/05/2026

NRC: 30705

1. Descripción del problema a resolver

El Sistema Ferroviario (SmartRail) resuelve los problemas derivados de la gestión manual y desorganizada de la información en el servicio de transporte ferroviario, tales como la dificultad para controlar reservas, usuarios, trenes en un solo entorno. Antes del sistema, la información se encontraba dispersa, lo que generaba errores, duplicidad de datos y demoras en la atención a los clientes. Además, la falta de automatización impedía conocer en tiempo real la disponibilidad de trenes y el estado de las reservas. También existía un control limitado de acceso, poniendo en riesgo la seguridad de la información. Este sistema centraliza todos los procesos, mejora la eficiencia operativa, reduce errores humanos y permite una administración más segura, ordenada y confiable del servicio ferroviario.

1.1 Objetivo del Proyecto

Diseñar e implementar un sistema de gestión denominado SmartRail que permita administrar los trenes, horarios y reservas, facilitando el control de la información y optimizando el tiempo del usuario.

1.2 Alcance

El alcance del proyecto comprende el diseño, desarrollo e implementación de una aplicación de escritorio orientada a centralizar y optimizar la gestión operativa del servicio ferroviario. El sistema estará desarrollado bajo el patrón de arquitectura Modelo-Vista-Controlador (MVC), utilizando el lenguaje Java (con interfaz gráfica en Java Swing) y MongoDB para la gestión eficiente de la base de datos.

- **Autenticación y Seguridad:** Control de acceso seguro mediante registro, inicio de sesión y asignación de roles.
- **Gestión de Reservas:** Búsqueda de trenes (por origen, destino y fecha), creación de reservas y cancelaciones.

- **Administración Operativa:** Control total de los trenes (creación, edición y horario) y gestión de las cuentas de usuario.
- **Reportes y Análisis:** Generación de informes estadísticos sobre la operatividad del sistema y las reservas para facilitar la toma de decisiones.

2. Modelo de procesos y metodología de trabajo

2.1 Modelo de proceso utilizado

El modelo de proceso utilizado en el desarrollo del sistema ferroviario (SmartRail) es el modelo incremental, con características iterativas. Esto se debe a que el sistema no se desarrolló completamente en una sola etapa, sino que se construyó por partes, agregando funcionalidades progresivamente como:

- Login de usuario
- Registro de usuario
- Control de roles (Cliente / Administrador)
- Panel de gestión

Cada funcionalidad se implementó, probó y mejoró antes de continuar con la siguiente.

2.2 Metodología aplicada

Metodología: Scrum

En el desarrollo del Sistema Ferroviario se aplicó una metodología ágil basada en Scrum, ya que el trabajo se organizó en ciclos cortos de desarrollo llamados sprints. Esto permitió construir el sistema de forma progresiva, desarrollando funcionalidades como el registro de usuarios, inicio de sesión y control de roles de manera independiente, validando cada una antes de continuar con la siguiente.

El proyecto se gestionó a través de historias de usuario organizadas en un backlog, lo que facilitó la planificación, priorización y control del avance. Además, se definieron roles dentro del equipo como Product Owner, Scrum Master y desarrolladores, permitiendo una adecuada distribución de responsabilidades y coordinación del trabajo.

Asimismo, se realizaron pruebas y validaciones continuas, como las pruebas de caja negra, con el objetivo de asegurar el correcto funcionamiento del sistema en cada etapa. En conjunto, esta metodología permitió mantener un desarrollo organizado, controlado y orientado a la mejora continua del sistema.

3.1 Cronograma de trabajo

[illegible]

REQUISITO	TEMA	COMO UN...	NECESITO	ASÍ PODRÉ...	NOTAS	PRIORIDAD	ESTATUS
REQ001	Autenticación de usuario	Administrador y Cliente	El programa debe tener una función para registrar y guardar datos de los clientes y administradores.	Autenticar los datos del administrador/cliente para iniciar sesión o registrar.	NA	Alta	Finalizado
	Tareas				Asignado		Estimado
REQ001-1	Enlazar el código con mongoDB.				Daniela Narváez		1
REQ001-2	Crear el registro de usuario.						2
REQ001-3	Validar datos						4
REQ001-4	Elaborar una interfaz gráfica para el registro e inicio de sesión con datos ya validados.						4
REQ001-5	Crear el controlador para validar los datos de la interfaz						3
REQ002	Administración de usuarios	Administrador	El programa debe permitir al administrador eliminar y editar datos de clientes.	Organizar los datos de los clientes o eliminarlos en caso de inactividad.	NA	Media	En proceso
	Tareas				Asignado		Estimado
REQ002-1	Un CRUD para el rol administrador				Daniela Narváez		4

4. Identificación de Requisitos

4.1 Requerimientos Funcionales

ID	Nombre del Requisito	Descripción
RF-01	Inicio de sesión y registro de usuario	Permitir que un cliente o administrador cree una cuenta en el sistema y acceso al sistema mediante credenciales válidas.
RF-02	Administración de clientes	Administrador: Podrá gestionar todos los usuarios que ingresen al sistema
RF-03	Gestionar Trenes	Crear, modificar o eliminar rutas (ya creadas) de los trenes.
RF-04	Gestionar Horario	Administrador: Podrá asignar y ver los horarios de salida y llegada cada tren del sistema
RF-05	Gestionar Reservas	Cliente: Tendrá la posibilidad de reservar boletos de tren, consultar el estado de sus reservas y cancelarlas dentro de los plazos permitidos.
RF-06	Generar Reportes	Administrador: Podrá generar los reportes del sistema de forma general o por reportes filtrados.

4.2 Requerimientos No Funcionales

ID	Nombre del Requisito	Descripción
RNF-01	Autenticar usuarios	El sistema debe cifrar las credenciales de acceso para que los datos personales estén protegidos contra accesos no autorizados
RNF-02	Elaborar una interfaz intuitiva	Diseño ergonómico y facilidad de navegación
RNF-03	Controlar acceso por roles	Restricción de funciones según el tipo de usuario
RNF-04	Validar datos únicos	Administrador: Podrá verificar con ayuda del sistema que no se asigne un código de tren que ya exista en la base de datos para evitar duplicidad.
RNF-05	Validar conflictos de horarios	Integridad en la planificación de rutas
RNF-06	Garantizar disponibilidad del sistema	El cliente tendrá la posibilidad de acceder al sistema para reservar boletos en cualquier momento del día sin interrupciones
RNF-07	Reducir tiempo de espera	El sistema debe garantizar que el cliente reciba la confirmación de su consulta o reserva de forma inmediata
RNF-08	Agilizar la generación de reportes	El sistema permitirá que el administrador obtenga los reportes filtrados de pagos y usuarios de manera ágil y sin demoras

4.3 Evidencias de toma de levantamiento de información

Archivo Inicio Insertar Disposición de página Fórmulas Datos Revisar Vista Ayuda Acrobat

calibri 10 A+ A- You are screen sharing Stop share

Portapapeles Pegar Fuente Alineación Número Estilos Celdas Edición

G6 1. B C D E F G H I J K

1 2 3 4 5 6

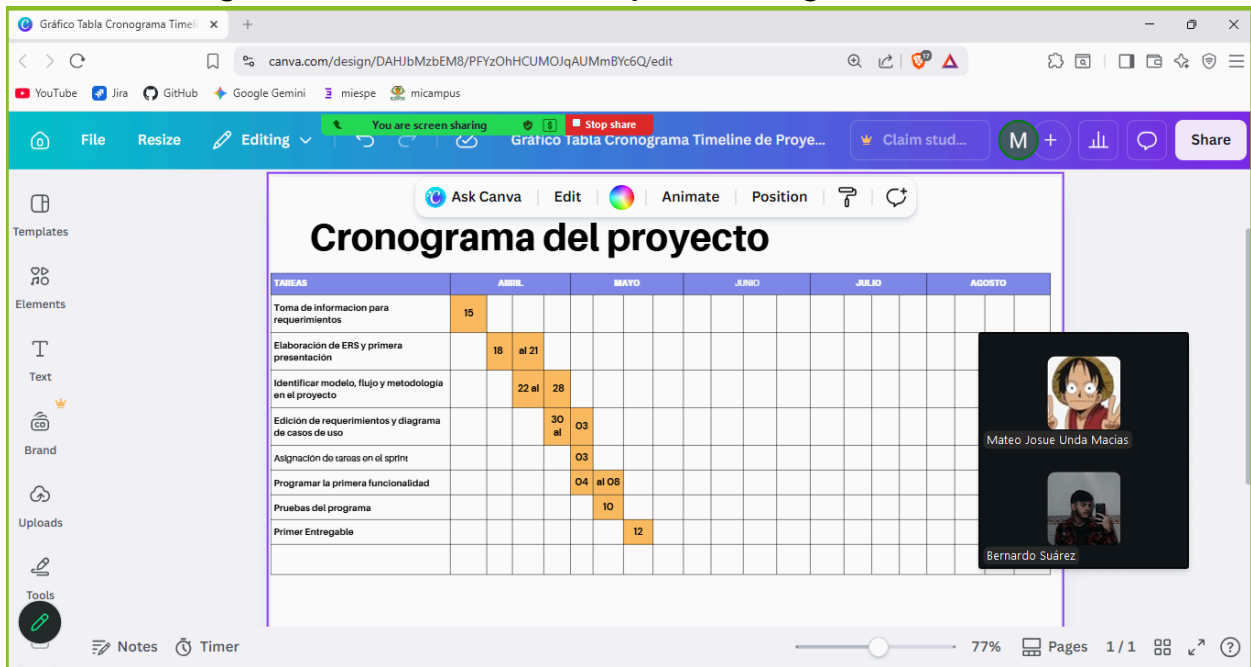
Matriz de Marco de Trabajo de HU

ITEM	PROBLEMA	QUE (NECESIDAD)	PARA QUE (SOLUCIÓN)	PARA QUIEN (USUARIO)	COMO (DESCRIPCIÓN DE TAREAS)	HECHO POR (PROG. RESP.)	CUANTO TIEMPO (EST. EN HRS)	FECHA DE ENTREGA	PRIORIDAD	STATUS	PRUEBA (COMO SE VERIFICA)	CC
REQ001	Los clientes y administradores no tienen acceso a las funcionalidades de SmartRail.	El programa debe tener una función de autenticación de usuarios.	Los usuarios se registrarán para facilitar el acceso en el futuro.	Administrador y Cliente	1.				Alta			

Formato descripción HU Historia de Usuario

Introducir Accesibilidad: es necesario investigar

Evidencia de organización del horario con el primer entregable



4.4 Relación entre requerimientos y funcionalidades del sistema

Funcionalidad Principal	Código	Descripción del Requerimiento Funcional
Autenticación y Roles	RF-01	El sistema debe permitir el registro, inicio de sesión seguro y la asignación de roles (Cliente o Administrador).
Gestión de Usuarios	RF-02	El sistema debe permitir al administrador gestionar las cuentas de los usuarios registrados (activar, bloquear o modificar información).
Gestión de reservas	RF-03	El sistema debe permitir al cliente crear una nueva reserva, visualizar su historial y realizar cancelaciones.
Administración de trenes	RF-04	El sistema debe permitir al administrador gestionar el registro físico de los trenes (crear, modificar, visualizar y eliminar unidades).
Programación de rutas	RF-05	El sistema debe permitir al administrador planificar la logística asignando y modificando horarios, orígenes y destinos de cada tren.
Reportes Operativos	RF-06	El sistema debe generar informes automáticos y estadísticas sobre el volumen de las reservas y la disponibilidad del sistema.

5. Casos de Uso

5.1 Identificar actores

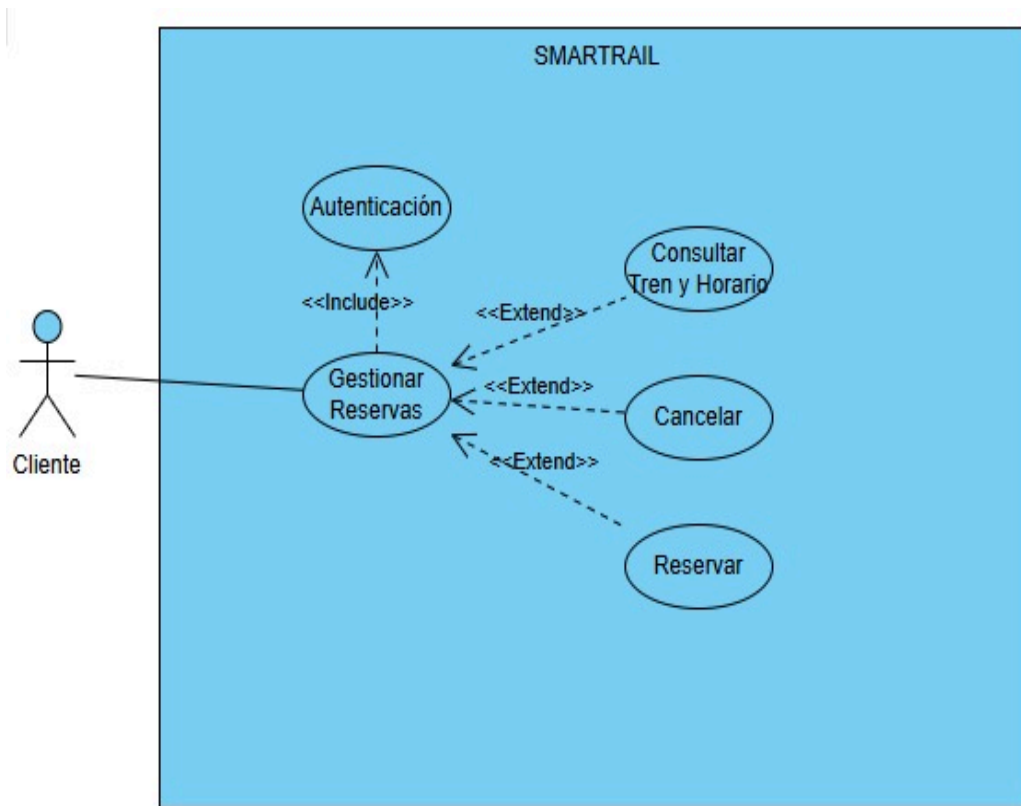
Actor 1

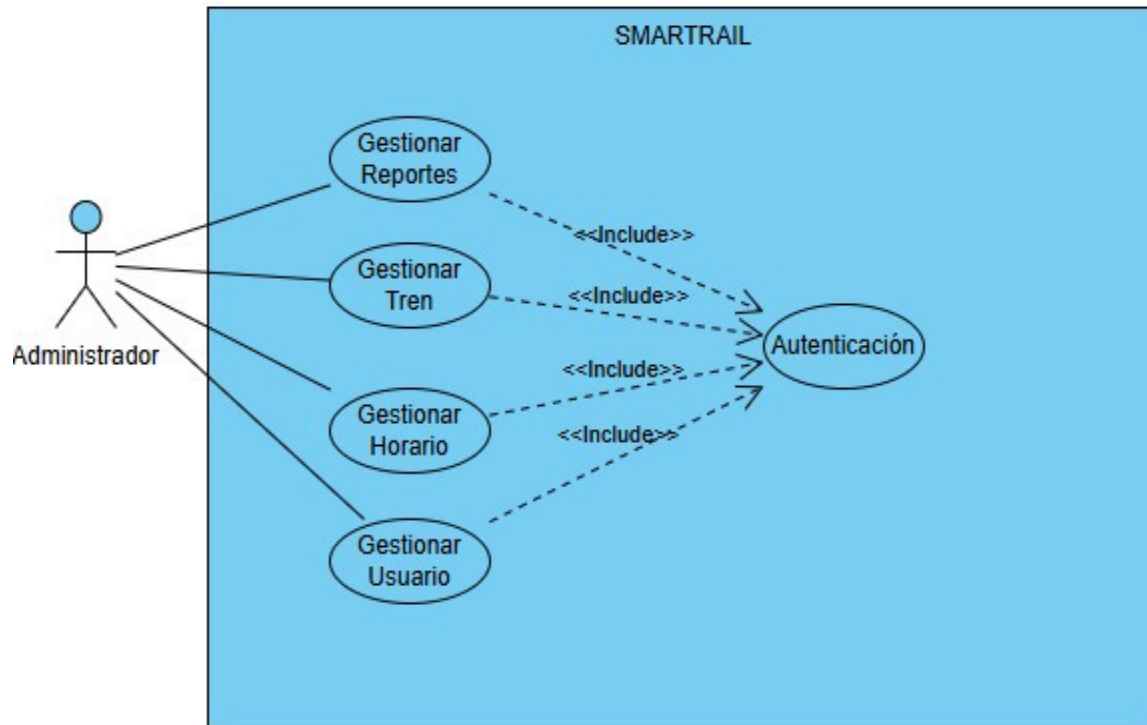
- Identificar actor: Cliente
- Identificar casos de uso:
 - Autenticación
 - Consultar tren y horario
 - Cancelar
 - Reservar

Actor 2

- Identificar actor: Administrador
- Identificar casos de uso:
 - Gestionar reportes
 - Gestionar tren
 - Gestionar horario
 - Gestionar usuario

5.2 Diagramas de casos de uso





5.3 Descripción de los casos

Actor 1

Caso 1

- Nombre : Autenticación
- Actor : Cliente
- Descripción : El cliente tendrá la posibilidad de crear un cuenta en el sistema para posteriormente iniciar sesion e ingresar a los diferentes módulos que tiene el sistema
- Flujo básico :
 - El cliente selecciona la opción de "Crear cuenta" en la pantalla inicial.
 - El cliente ingresa los datos solicitados por el formulario.
 - El sistema valida que el correo electrónico no esté registrado previamente.
 - El sistema crea la cuenta, almacena la información y confirma el registro exitoso.
 - El cliente ingresa sus credenciales (correo y contraseña) en la pantalla de inicio.
 - El sistema valida los datos y permite el ingreso al panel principal.

Caso 2

- Nombre : Consultar tren y horario
- Actor : Cliente
- Descripción : El cliente podrá ver los horarios y rutas que tiene los trenes que están operando
- Flujo básico :
 - El cliente accede al módulo de consultas.

- El cliente selecciona un filtro de búsqueda (ej. por fecha).
- El sistema despliega los resultados con el detalle de cada tren (ID, código, fecha, salida, llegada, capacidad, precio y estado)

Caso 3

- Nombre : Cancelar
- Actor : Cliente
- Descripción : El cliente podrá cancelar una reserva
- Flujo básico :
 - El cliente accede al módulo de Cancelar
 - El sistema muestra la lista de reservas activas vinculadas a su cuenta.
 - El cliente selecciona la opción "Cancelar" en la reserva deseada.
 - El sistema muestra un mensaje confirmando que la cancelación se ha completado.
 - El sistema actualiza la base de datos y cancela la reserva de la lista del cliente.

Caso 4

- Nombre : Reservar
- Actor : Cliente
- Descripción : El cliente podrá reservar boletos de viaje
- Flujo básico :
 - El cliente accede al módulo de reservas.
 - El sistema muestra los trenes disponibles.
 - El cliente selecciona el tren y el horario deseado.
 - El sistema valida la disponibilidad.
 - El sistema registra la reserva y muestra un mensaje de confirmación.

Actor 2

Caso 1

- Nombre : Autenticación
- Actor : Administrador
- Descripción : El administrador debe de ingresar sus credenciales para ingresar al sistema
- Flujo básico :
 - El administrador accede a la pantalla de inicio
 - El Administrador ingresa sus credenciales (Nombre, Usuario y contraseña)
 - El sistema valida las credenciales y permite el ingreso al sistema
 - El sistema permite el acceso y redirige al panel principal

Caso 2

- Nombre : Gestionar Usuario
- Actor : Administrador
- Descripción : El administrador tendrá acceso a una lista donde pueda ver los clientes que tiene el sistema los cuales podrá modificar o eliminar
- Flujo básico :
 - El administrador accede al módulo de Usuario
 - El sistema despliega la lista completa de clientes registrados.
 - El sistema muestra las opciones de "Editar" y "Eliminar" junto a cada registro
 - El administrador selecciona la acción deseada

Caso 2.1 (Flujo de eliminación)

- El administrador selecciona la opción "Eliminar" de un cliente específico.
- El sistema solicita una confirmación para proceder con el borrado.
- El administrador acepta la confirmación.
- El sistema elimina el registro y actualiza la lista en pantalla.

caso 2.2 (Flujo de edición)

- El administrador selecciona la opción "Editar" de un cliente específico.
- El sistema despliega un formulario con los datos actuales del usuario.
- El administrador modifica los campos necesarios.
- El administrador selecciona la opción "Aceptar" o "Guardar".
- El sistema valida los cambios, actualiza la base de datos y muestra los nuevos datos.

Caso 3

- Nombre : Gestionar Horario
- Actor : Administrador
- Descripción : Podrá administrar la información de los trenes (Crear, modificar y eliminar rutas)
- Flujo básico :
 - El sistema mostrará en pantalla el horario de cada tren y una opción de asignar
 - El Administrador selecciona la opción de asignar
 - El sistema despliega una pantalla donde se debe llenar los horarios del tren
 - El administrador asigna los horarios del tren
 - El administrador confirma esta asignación
 - El sistema muestra en pantalla los horarios del tren

Caso 4

- Nombre : Gestionar tren
- Actor : Administrador
- Descripción : Podrá administrar la información de los trenes (Crear, modificar y eliminar rutas)
- Flujo básico :
 - El administrador accede al módulo de Gestión de Trenes/Rutas.

- El sistema muestra la lista de rutas y trenes registrados con opciones para crear, modificar o eliminar.
- El administrador selecciona la acción deseada (Crear, Editar o Eliminar)
- El sistema despliega el formulario correspondiente para ingresar o actualizar los datos
- El administrador ingresa la información de la ruta y confirma la operación
- El sistema valida los datos, guarda los cambios y actualiza la lista en pantalla

Caso 5

- Nombre : Gestionar Reportes
- Actor : Administrador
- Descripción : Podrá generar los reportes del sistema de forma general o por reportes filtrados.
- Flujo básico :
 - El administrador accede al módulo de Reportes
 - El sistema muestra los tipos de reportes disponibles y las opciones de filtrado (fusuario, tren, ruta, etc.).
 - El administrador selecciona el tipo de reporte y aplica los filtros si es necesario.
 - El administrador selecciona la opción de "Generar Reporte".
 - El sistema procesa la solicitud y el sistema muestra el reporte en pantalla

6. Evidencias del proceso de desarrollo

6.1 Reuniones de trabajo

The screenshot displays a Zoom meeting in progress. The main window shows a screen share of an Excel spreadsheet titled "BACKLOG.1 - Excel". The spreadsheet is organized into sections for different tasks, each with a "Tareas" (Tasks) header. The tasks are listed with their IDs, descriptions, assignees, and status.

ID	Tarea	Asignado	Estado
REQ001	Autenticación de usuario	NA	En proceso
REQ001-1	Enlazar el código con mongoDB.	Daniela Narváez	1
REQ001-2	Crear el registro de usuario.		2
REQ001-3	Validar datos		4
REQ001-4	Elaborar una interfaz gráfica para el registro e inicio de sesión con datos ya validados.		4
REQ001-5	Crear el controlador para validar los datos de la interfaz		3
REQ002	Administración de usuarios	NA	En proceso
REQ002-1	Un CRUD para el rol administrador	Daniela Narváez	4
REQ003	Gestionar trenes	NA	En proceso
REQ003-1	Crear un registro de trenes	Mateo Unda	2
REQ003-2	Elaborar una interfaz gráfica que cree, modifique y elimine rutas y trenes		4

The Zoom sidebar on the right shows the following participants:

- Mateo Josue Unda Macias
- JOSE GABRIEL VANEZ CEPEDA
- Daniela
- Anthony Vilaña

The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 23:14 on 10/5/2026.

Screenshot of a Google Drive document titled "5 CMMI.pdf" showing a project plan for a system. The document is viewed in a web browser with a sidebar on the left showing navigation options like "Nuevo", "Página principal", "Actividad", etc. The main content area displays sections 3, 4, and 5 of the document, each marked with a red checkmark. Section 3, "IDENTIFICACIÓN DEL REQUISITO", includes a table with columns "ID", "Nombre", and "Descripción". Section 4, "PLANIFICACIÓN (Relación con Nivel 2 - Gestionado)", includes a table with columns "Elemento" and "Detalle". Section 5, "ANÁLISIS Y DISEÑO (Relación con Nivel 3 - Definido)", is partially visible. The document is shared with several users, including Mateo Josue Unda Macias, JOSE GABRIEL YANEZ CEPEDA, Daniela, and Anthony Vilaña. The bottom of the screenshot shows a Windows taskbar with various application icons and a system clock indicating 23:15 on 10/5/2026.

6.2 Historias de Usuario

ITEM	PROBLEMA	QUE NECESIDAD	PARA QUE SOLUCIÓN	PARA QUE/EN (USUARIO)	CÓMO (DESCRIPCIÓN DE TAREAS)	CUANTO TIEMPO (ESTIMADO EN HRS)	FECHA DE ENTREGA	PRIORIDAD	STATUS	PRUEBA (CÓMO SE VERIFICA)	COMENTARIOS	NOMBRE DE HISTORIA
REQ001	La acumulación de cuentas inactivas y datos desactualizados genera desorden en la base de datos, dificultando la gestión de la información de los usuarios activos.	El programa debe tener una función para registrar y guardar datos de los clientes administradores.	Autenticar los datos del administrador/usuario para iniciar sesión o registrar.	Administrador y usuario	1. Entrar el código con mongoDB. 2. Crear el registro de usuario. 3. Validar datos. 4. Elaborar una interfaz gráfica para el registro e inicio de sesión con datos ya validados. 5. Crear el controlador para validar los datos de la interfaz.	6h	5/12/2026	Alta	Terminado	1. Ingresar con credenciales de administrador y verificar acceso al panel de control. 2. Ingresar con credenciales de cliente y verificar que no se tenga acceso a herramientas administrativas. 3. Ingresar credenciales incorrectas y validar que el sistema muestre un mensaje de error y deniegue el acceso.		Autenticación de usuarios con roles diferenciados
REQ002	La acumulación de cuentas inactivas y datos desactualizados genera desorden en la base de datos, dificultando la gestión de la información de los usuarios activos.	El programa debe permitir al administrador eliminar y editar datos de clientes.	Organizar los datos de los clientes o eliminarlos en caso de inactividad.	Administrador	1. Un CRUD para el rol administrador	6h	por definir	Meda	En proceso	1. Seleccionar un cliente existente, modificar un dato y guardar, verificando que el cambio se refleje. 2. Seleccionar un cliente inactivo y eliminar su registro, comprobando que ya no aparezca en la lista del sistema.		Administración de usuarios
REQ003	No existe un mecanismo centralizado para registrar, nuevas rutas o modificar las existentes frente a cambios operativos, lo que limita la expansión o ajuste del servicio de transporte.	El programa debe permitir al administrador crear, modificar y eliminar rutas planificadas.	Administrar los próximos movimientos de los trenes y eliminarlos en caso de eventualidades.	Administrador	1. Crear un registro de trenes 2. Elaborar una interfaz gráfica que cree, modifique y elimine rutas y trenes. 3. Desarrollar un controlador para la interfaz gráfica.	6h	por definir	Alta	En proceso	1. Crear una nueva ruta de tren llenando todos los campos requeridos y verificar que se guarde exitosamente. 2. Editar la ruta creada previamente. 3. Hacer clic en eliminar ruta y verificar que el sistema solicite una confirmación antes de borrar el registro definitivamente.		Gestionar Trenes
REQ004	La rigidez en la planificación actual impide reaccionar y actualizar la información de manera ágil cuando ocurren retrasos, reprogramaciones o cancelación de viajes por imprevistos.	El programa debe permitir que el administrador organice los horarios de los trenes.	Organizar los horarios de los trenes, permitiendo editar o eliminar un viaje en caso de retrasos o problemas de los trenes.	Administrador	1. Implementar servicio de calendarización 2. Elaborar una interfaz gráfica que cree, modifique y elimine horarios. 3. Desarrollar un controlador para la interfaz gráfica.	6h	por definir	Alta	En proceso	1. Asignar un horario específico a un tren y validar que se publique correctamente. 2. Simular un retraso modificando la hora de salida de un viaje programado y verificar que el cambio se guarde y actualice. 3. Eliminar un horario y confirmar que el viaje desaparezca de la planificación.		Gestionar horarios
REQ005	Los pasajeros dependen de procesos manuales para adquirir, consultar o anular sus pasajes, lo que genera cuellos de botella y una mala experiencia de usuario al no tener autonomía.	El programa debe permitir que el cliente pueda hacer reservas de boletos, consultar su estado y cancelarlo de ser necesario.	Ver los horarios de trenes más convenientes, consultar su estado y cancelarlo con su respectiva anticipación.	Cliente	1. Implementar el servicio de gestión de reservas de clientes. 2. Elaborar una interfaz gráfica que enseñe al cliente el estado de su reserva. 3. Desarrollar un controlador para la interfaz gráfica.	6h	por definir	Alta	En proceso	1. Iniciar sesión como cliente, seleccionar una ruta disponible y completar una reserva exitosa. 2. Ir a la sección de "Mis Reservas" y validar que el estado del boleto sea correcto. 3. Ejecutar la opción de cancelar la reserva y verificar que el asiento del tren vuelva a estar disponible en el sistema.		Gestionar Reservas
REQ006	Los administradores no tienen visibilidad de las métricas de uso, estado de trenes y volumen de reservas, lo que dificulta el análisis de datos y la toma de decisiones estratégicas.	El programa debe permitir que el administrador pueda generar los reportes del sistema de forma general o por reportes filtrados.	Ver los reportes del sistema, para una mejor información del movimiento de los trenes.	Administrador	1. Imprimir los trenes, horarios y reservas 2. Crear una interfaz gráfica para los reportes. 3. Desarrollar un controlador para la interfaz gráfica.	7h	por definir	Meda	En proceso	1. Generar un reporte general y verificar que la tabla muestre la información de forma estructurada. 2. Navegar mediante la paginación para asegurar que grandes volúmenes de datos carguen correctamente sin saturar la pantalla.		Gestionar reportes

6.3 Backlog

REQUISITO	TEMA	COMO UN...	NECESITO	ASÍ PODRÉ...	NOTAS	PRIORIDAD	ESTATUS
REQ001	Autenticación de usuario	Administrador y Cliente	El programa debe tener una función para registrar y guardar datos de los clientes y administradores.	Autenticar los datos del administrador/cliente para iniciar sesión o registrar.	NA	Alta	Terminado
REQ002	Administración de usuarios	Administrador	El programa debe permitir al administrador eliminar y editar datos de clientes.	Organizar los datos de los clientes o eliminarlos en caso de inactividad.	NA	Media	En proceso
REQ003	Gestionar trenes	Administrador	El programa debe permitir al administrador crear, modificar o eliminar rutas planificadas de los trenes.	Administrar los próximos movimientos de los trenes y cambiarlos en caso de eventualidades.	NA	Alta	En proceso
REQ004	Gestionar horarios	Administrador	El programa debe permitir que el administrador organice los horarios de los trenes.	Organizar los horarios de los trenes, permitiendo editar o eliminar un viaje en caso de retrasos o problemas de los trenes	NA	Alta	En proceso
REQ005	Gestionar reservas	Cliente	El programa debe permitir que el cliente pueda hacer reservas de boletos, consultar su estado y cancelarlo de ser necesario.	Ver los horarios de trenes mas convenientes, consultar su estado y cancelarlo con su respectiva anticipación.	NA	Alta	En proceso
REQ006	Gestionar reportes	Administrador	El programa debe permitir que el administrador pueda generar los reportes del sistema de forma general o por reportes filtrados.	Ver los reportes del sistema, para una mejor información del movimiento de los trenes.	NA	Media	En proceso

6.4 Avances del sistema

Paquete Modelo

Conexión a MongoDB

```
package modelo;
```

```
import com.mongodb.client.MongoClient;
```

```
import com.mongodb.client.MongoClients;
```

```
import com.mongodb.client.MongoDatabase;
```

```
// Patrón Singleton: garantiza una única conexión a MongoDB en toda la aplicación.
```

```
// SOLID (S): responsabilidad única — solo gestiona la conexión a la base de datos.
```

```
public final class ConexionBD {
```

```
    private static final String URI = "mongodb://localhost:27017";
```

```
    private static final String DATABASE_NAME = "ferroviaria";
```

```
    // Singleton: instancia única del cliente MongoDB
```

```
    private static MongoClient mongoClient;
```

```
    // Singleton: constructor privado impide instanciación externa
```

```
    private ConexionBD() {
```

```

    }

    // Singleton: punto de acceso global con inicialización perezosa (lazy init)
    public static MongoDBDatabase getConexion() {
        if (mongoClient == null) {
            mongoClient = MongoClients.create(URI);
        }
        return mongoClient.getDatabase(DATABASE_NAME);
    }
}

```

Control Sesión

```

package modelo;

// Patrón Singleton: mantiene una única sesión de usuario activa en toda la aplicación.
// SOLID (S): responsabilidad única — solo gestiona el estado de la sesión del usuario.
public final class ControlSesion {

    // Singleton: instancia estática única del usuario en sesión
    private static Usuario usuarioActual;

    // Singleton: constructor privado impide instanciación externa
    private ControlSesion() {
    }

    public static void iniciarSesion(Usuario u) {
        usuarioActual = u;
    }

    public static Usuario getUsuario() {
        return usuarioActual;
    }

    public static void cerrarSesion() {

```



```

        usuarioActual = null;
    }

    public static boolean sesionActiva() {
        return usuarioActual != null;
    }

    public static boolean esAdmin() {
        return usuarioActual != null
            && usuarioActual.getRol() != null
            && usuarioActual.getRol().equalsIgnoreCase("Admin");
    }
}

```

Usuario

```

package modelo;

// SOLID (S): responsabilidad única — solo representa los datos de un usuario.

public class Usuario {
    private String id;
    private String nombre;
    private String username;
    private String password;
    private String rol;
    private boolean activo;

    public Usuario() {
    }

    public String getId() {
        return id;
    }
}

```

```
public void setId(String id) {  
    this.id = id;  
}  
  
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public String getUsername() {  
    return username;  
}  
  
public void setUsername(String username) {  
    this.username = username;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
  
public String getRol() {  
    return rol;  
}  
  
public void setRol(String rol) {  
    this.rol = rol;  
}
```

```

    }

    public boolean isActive() {
        return activo;
    }

    public void setActivo(boolean activo) {
        this.activo = activo;
    }
}

```

Usuario DAO

```

package modelo;

import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import static com.mongodb.client.model.Filters.*;
import java.util.ArrayList;
import java.util.List;
import org.bson.Document;

// Patrón DAO: encapsula el acceso a la colección "usuarios" de MongoDB.
// SOLID (S): responsabilidad única — solo gestiona la persistencia de usuarios.

public class UsuarioDAO {
    private MongoCollection<Document> coleccion;

    public UsuarioDAO() {
        MongoDatabase db = ConexionBD.getConexion();
        if (db != null) {
            coleccion = db.getCollection("usuarios");
        }
    }
}

```

```

}

public Usuario login(String username, String password) {
    if (coleccion == null
        || username == null || username.trim().isEmpty()
        || password == null || password.trim().isEmpty()) {
        return null;
    }

    Document doc = coleccion.find(and(
        eq("username", username.trim()),
        eq("activo", true))).first();

    if (doc != null) {
        String passAlmacenado = doc.getString("password");

        if (password.equals(passAlmacenado)) {
            return documentToUsuario(doc);
        }
    }

    return null;
}

public boolean registrar(Usuario u) {
    if (coleccion == null || u == null) {
        return false;
    }

    if (u.getUsername() == null || u.getUsername().trim().isEmpty()) {
        return false;
    }
}

```

```

    if (u.getPassword() == null || u.getPassword().trim().isEmpty()) {
        return false;
    }
    if (coleccion.find(eq("username", u.getUsername().trim())).first() != null) {
        return false;
    }
    Document doc = new Document("nombre", u.getNombre())
        .append("username", u.getUsername().trim())
        .append("password", u.getPassword())
        .append("rol", u.getRol())
        .append("activo", true);
    coleccion.insertOne(doc);
    return true;
}

public List<Usuario> listarUsuarios() {
    List<Usuario> lista = new ArrayList<>();
    if (coleccion == null) {
        return lista;
    }
    for (Document doc : coleccion.find()) {
        lista.add(documentToUsuario(doc));
    }
    return lista;
}

private Usuario documentToUsuario(Document doc) {
    Usuario u = new Usuario();

```

```

        if (doc.getObjectId("_id") != null) {
            u.setId(doc.getObjectId("_id").toHexString());
        }
        u.setNombre(doc.getString("nombre"));
        u.setUsername(doc.getString("username"));
        u.setRol(doc.getString("rol"));
        u.setActivo(doc.getBoolean("activo", true));
        return u;
    }
}

```

Validador de datos

```

package modelo;

// SOLID (S): responsabilidad única — centraliza las validaciones de datos reutilizables.

public final class ValidadorDatos {

    private ValidadorDatos() {

    }

    public static boolean textoVacio(String texto) {
        return texto == null || texto.trim().isEmpty();
    }

    public static boolean longitudMinima(String texto, int min) {
        return texto != null && texto.trim().length() >= min;
    }

    public static boolean numeroPositivo(double valor) {
        return valor > 0;
    }
}

```

```
}
```

Paquete Vista

FrmPrincipal

```
package vista;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.Insets;
import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import modelo.ControlSesion;

// Patrón MVC: ventana principal que carga paneles (vistas) según la navegación del usuario.
// SOLID (S): responsabilidad única — solo gestiona la navegación y el layout principal.

public class FrmPrincipal extends javax.swing.JFrame {

    private static final int SIDEBAR_WIDTH = 180;

    private static final int MIN_WIDTH = 820;

    private static final int MIN_HEIGHT = 520;

    public FrmPrincipal() {

        initComponents();

        construirSidebar();
    }
}
```

```

setLocationRelativeTo(null);

setMinimumSize(new Dimension(MIN_WIDTH, MIN_HEIGHT));

if (!ControlSesion.sesionActiva()) {
    dispose();
    new FrmLogin().setVisible(true);
    return;
}

boolean esAdmin = ControlSesion.esAdmin();

if (!esAdmin) {
    itemUsuarios.setVisible(false);
    itemReportes.setVisible(false);
    itemRegistroTren.setVisible(false);
    btnUsuarios.setVisible(false);
    btnReportes.setVisible(false);
    btnRegistroTren.setVisible(false);
}
}

private void construirSidebar() {
    pnlSidebar.setBackground(TemaSistema.getFondoCard());
    pnlSidebar.setLayout(new BoxLayout(pnlSidebar, BoxLayout.Y_AXIS));
    pnlSidebar.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createMatteBorder(0, 0, 0, 1, new Color(0xE0E0E0)),
        BorderFactory.createEmptyBorder(16, 8, 16, 8)));
    Insets insets = new Insets(10, 20, 10, 20);
    Font btnFont = new Font(Font.SANS_SERIF, Font.PLAIN, 14);
    btnReservas = crearBotonSidebar("Reservas", insets, btnFont);

```



```

    btnTrenes = crearBotonSidebar("Trenes", insets, btnFont);
    btnUsuarios = crearBotonSidebar("Usuarios", insets, btnFont);
    btnReportes = crearBotonSidebar("Reportes", insets, btnFont);
    btnRegistroTren = crearBotonSidebar("Registrar tren", insets, btnFont);
    btnSalir = crearBotonSidebar("Salir", insets, btnFont);
    btnSalir.setForeground(new Color(0xC62828));
    btnUsuarios.addActionListener(e -> cargarPanel(new PnlUsuario()));
    btnSalir.addActionListener(this::confirmarSalir);
    pnlSidebar.add(Box.createVerticalStrut(8));
    pnlSidebar.add(btnReservas);
    pnlSidebar.add(Box.createVerticalStrut(6));
    pnlSidebar.add(btnTrenes);
    pnlSidebar.add(Box.createVerticalStrut(6));
    pnlSidebar.add(btnUsuarios);
    pnlSidebar.add(Box.createVerticalStrut(6));
    pnlSidebar.add(btnReportes);
    pnlSidebar.add(Box.createVerticalStrut(6));
    pnlSidebar.add(btnRegistroTren);
    pnlSidebar.add(Box.createVerticalGlue());
    pnlSidebar.add(btnSalir);
}

private JButton crearBotonSidebar(String texto, Insets insets, Font font) {
    JButton b = new JButton(texto);
    b.setMaximumSize(new Dimension(SIDEBAR_WIDTH - 16, 44));
    b.setAlignmentX(java.awt.Component.CENTER_ALIGNMENT);
    b.setMargin(insets);
}

```

```

        b.setFont(font);

        b.setBackground(TemaSistema.getFondoCard());

        b.setForeground(TemaSistema.getTexto());

        b.setBorder(BorderFactory.createEmptyBorder(10, 20, 10, 20));

        b.setFocusPainted(false);

        b.setContentAreaFilled(true);

        return b;
    }

    private void confirmarSalir(java.awt.event.ActionEvent evt) {

        int op = JOptionPane.showConfirmDialog(this, "¿Cerrar sesión?", "Salir",
        JOptionPane.YES_NO_OPTION);

        if (op == JOptionPane.YES_OPTION) {

            ControlSesion.cerrarSesion();

            dispose();

            new FrmLogin().setVisible(true);

        }

    }

    private void cargarPanel(JPanel panel) {

        pnlContenido.removeAll();

        pnlContenido.setLayout(new BorderLayout());

        pnlContenido.add(panel, BorderLayout.CENTER);

        pnlContenido.setBackground(TemaSistema.getFondo());

        pnlContenido.revalidate();

        pnlContenido.repaint();

    }

    @SuppressWarnings("unchecked")

```

```
// <editor-fold defaultstate="collapsed" desc="Generated
// Code">

private void initComponents() {

    pnlSidebar = new javax.swing.JPanel();

    pnlSidebar.setPreferredSize(new Dimension(SIDEBAR_WIDTH, 400));

    pnlContenido = new javax.swing.JPanel();

    pnlContenido.setBackground(TemaSistema.getFondo());

    pnlContenido.setLayout(new BorderLayout());

    menuBar = new javax.swing.JMenuBar();

    menuSistema = new javax.swing.JMenu();

    itemUsuarios = new javax.swing.JMenuItem();

    itemTrenes = new javax.swing.JMenuItem();

    itemReservas = new javax.swing.JMenuItem();

    itemReportes = new javax.swing.JMenuItem();

    itemRegistroTren = new javax.swing.JMenuItem();

    itemSalir = new javax.swing.JMenuItem();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    setTitle("SmartRail");

    getContentPane().setBackground(TemaSistema.getFondo());

    menuSistema.setText("Menú");

    itemUsuarios.setText("Usuarios");

    itemUsuarios.addActionListener(this::itemUsuariosActionPerformed);

    menuSistema.add(itemUsuarios);

    itemTrenes.setText("Trenes");

    menuSistema.add(itemTrenes);

    itemReservas.setText("Reservas");
```

```

menuSistema.add(itemReservas);

itemReportes.setText("Reportes");

menuSistema.add(itemReportes);

itemRegistroTren.setText("Registrar Tren");

menuSistema.add(itemRegistroTren);

itemSalir.setText("Salir");

itemSalir.addActionListener(this::itemSalirActionPerformed);

menuSistema.add(itemSalir);

menuBar.add(menuSistema);

setJMenuBar(menuBar);


javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);

layout.setHorizontalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addComponent(pnlSidebar, javax.swing.GroupLayout.PREFERRED_SIZE,
SIDEBAR_WIDTH,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(0, 0, 0)
            .addComponent(pnlContenido, javax.swing.GroupLayout.DEFAULT_SIZE,
644,
                Short.MAX_VALUE)));

layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addComponent(pnlSidebar, javax.swing.GroupLayout.DEFAULT_SIZE,
                javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

```

```

        .addComponent(pnlContenido, javax.swing.GroupLayout.DEFAULT_SIZE,
            javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE));

    pack();
} // </editor-fold>

private void itemUsuariosActionPerformed(java.awt.event.ActionEvent evt) {
    cargarPanel(new PnlUsuario());
}

private void itemSalirActionPerformed(java.awt.event.ActionEvent evt) {
    confirmarSalir(evt);
}

public static void main(String[] args) {
    java.awt.EventQueue.invokeLater(() -> {
        TemaSistema.aplicar();
        new FrmPrincipal().setVisible(true);
    });
}

// Variables declaration - do not modify
private javax.swing.JMenuItem itemRegistroTren;
private javax.swing.JMenuItem itemReportes;
private javax.swing.JMenuItem itemReservas;
private javax.swing.JMenuItem itemSalir;

```

```
private javax.swing.JMenuItem itemTrenes;
private javax.swing.JMenuItem itemUsuarios;
private javax.swing.JMenuBar menuBar;
private javax.swing.JMenu menuSistema;
private javax.swing.JPanel pnlContenido;
private javax.swing.JPanel pnlSidebar;
private JButton btnReservas;
private JButton btnTrenes;
private JButton btnUsuarios;
private JButton btnReportes;
private JButton btnRegistroTren;
private JButton btnSalir;

// End of variables declaration
}
```

FrmUsuario

```
package vista;

import controlador.UsuarioControlador;
import java.util.HashMap;
import java.util.Map;
import javax.swing.JOptionPane;
import javax.swing.SwingUtilities;
import modelo.Usuario;
import util.EstrategiaValidacion;
import util.ValidacionUsuario;
```

```
public class PnlUsuario extends javax.swing.JPanel {

    private boolean modoRegistro;

    public PnlUsuario() {
        initComponents();
        aplicarTema();
    }

    public PnlUsuario(boolean modoRegistro) {
        initComponents();
        aplicarTema();
        this.modoRegistro = modoRegistro;

        if (modoRegistro) {
            cboRol.setSelectedItem("Cliente");
            cboRol.setEnabled(false); // puedes cambiar a true si quieres permitir admin
        } else {
            if (!modelo.ControlSesion.getUsuario().getRol().equalsIgnoreCase("ADMIN")) {
                JOptionPane.showMessageDialog(this, "Acceso restringido a Administradores");
                txtNombre.setEnabled(false);
                txtUsuario.setEnabled(false);
                txtPassword.setEnabled(false);
                cboRol.setEnabled(false);
                btnGuardar.setEnabled(false);
                btnLimpiar.setEnabled(false);
            }
        }
    }
}
```

```
    }  
    }  
}
```

```
private void aplicarTema() {  
    setBackground(TemaSistema.getFondo());  
    jPanel1.setBackground(TemaSistema.getFondo());  
  
    setBorder(javax.swing.BorderFactory.createEmptyBorder(  
        TemaSistema.getPadding(),  
        TemaSistema.getPadding(),  
        TemaSistema.getPadding(),  
        TemaSistema.getPadding()  
    ));  
}
```

```
@SuppressWarnings("unchecked")
```

```
private void initComponents() {  
  
    jPanel1 = new javax.swing.JPanel();  
    lblTitulo = new javax.swing.JLabel();  
    lblNombre = new javax.swing.JLabel();  
    lblCorreo = new javax.swing.JLabel();  
    lblContraseña = new javax.swing.JLabel();  
    lblRol = new javax.swing.JLabel();  
}
```



```
txtNombre = new javax.swing.JTextField();  
txtUsuario = new javax.swing.JTextField();  
txtPassword = new javax.swing.JPasswordField();  
cboRol = new javax.swing.JComboBox<>();
```

```
btnGuardar = new javax.swing.JButton();  
btnLimpiar = new javax.swing.JButton();
```

```
lblTitulo.setText("Registro de usuarios");  
lblTitulo.setFont(lblTitulo.getFont().deriveFont(java.awt.Font.BOLD, 16f));
```

```
lblNombre.setText("Nombre:");  
lblCorreo.setText("Usuario:");  
lblContraseña.setText("Contraseña:");  
lblRol.setText("Rol:");
```

```
txtNombre.setColumns(22);  
txtUsuario.setColumns(22);  
txtPassword.setColumns(22);
```

```
cboRol.setModel(new javax.swing.DefaultComboBoxModel<>(  
    new String[]{"Cliente", "Administrador"}));
```

```
btnGuardar.setText("Guardar");  
btnLimpiar.setText("Limpiar");
```

```
btnGuardar.addActionListener(this::btnGuardarActionPerformed);
```

```
btnLimpiar.addActionListener(this::btnLimpiarActionPerformed);
```

```
javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);
```

```
jPanel1.setLayout(jPanel1Layout);
```

```
jPanel1Layout.setHorizontalGroup(
```

```
    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
        .addComponent(lblTitulo)
```

```
        .addGroup(jPanel1Layout.createSequentialGroup()
```

```
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
                .addComponent(lblNombre, 90, 90, 90)
```

```
                .addComponent(lblCorreo)
```

```
                .addComponent(lblContraseña)
```

```
                .addComponent(lblRol))
```

```
            .addGap(10)
```

```
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
                .addComponent(txtNombre)
```

```
                .addComponent(txtUsuario)
```

```
                .addComponent(txtPassword)
```

```
                .addComponent(cboRol, 0, 220, Short.MAX_VALUE)))
```

```
            .addGroup(jPanel1Layout.createSequentialGroup()
```

```
                .addComponent(btnGuardar)
```

```

        .addGap(10)
        .addComponent(btnLimpiar))
    );

    jPanel1Layout.setVerticalGroup(
        jPanel1Layout.createSequentialGroup()
            .addComponent(lblTitulo)
            .addGap(20)

            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(lblNombre)
                .addComponent(txtNombre, javax.swing.GroupLayout.PREFERRED_SIZE,
                    javax.swing.GroupLayout.DEFAULT_SIZE,
                    javax.swing.GroupLayout.PREFERRED_SIZE))
            .addGap(10)

            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(lblCorreo)
                .addComponent(txtUsuario, javax.swing.GroupLayout.PREFERRED_SIZE,
                    javax.swing.GroupLayout.DEFAULT_SIZE,
                    javax.swing.GroupLayout.PREFERRED_SIZE))
            .addGap(10)

            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(lblContraseña)
                .addComponent(txtPassword,
                    javax.swing.GroupLayout.PREFERRED_SIZE,
                    javax.swing.GroupLayout.DEFAULT_SIZE,

```

```

        javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(10)

        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(lblRol)
            .addComponent(cboRol, javax.swing.GroupLayout.PREFERRED_SIZE,
                javax.swing.GroupLayout.DEFAULT_SIZE,
                javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(20)

        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(btnGuardar)
            .addComponent(btnLimpiar))

    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(this);
    this.setLayout(layout);

    layout.setHorizontalGroup(
        layout.createParallelGroup().addComponent(jPanel1)
    );

    layout.setVerticalGroup(
        layout.createParallelGroup().addComponent(jPanel1)
    );
}

```

```
private void btnGuardarActionPerformed(java.awt.event.ActionEvent evt) {  
  
    String nombre = txtNombre.getText().trim();  
    String usuario = txtUsuario.getText().trim();  
    String password = new String(txtPassword.getPassword());  
    String rolSeleccionado = cboRol.getSelectedItem().toString();  
  
    EstrategiaValidacion estrategia = new ValidacionUsuario();  
    Map<String, Object> datos = new HashMap<>();  
    datos.put("nombre", nombre);  
    datos.put("username", usuario);  
    datos.put("password", password);  
  
    String error = estrategia.validar(datos);  
    if (error != null) {  
        JOptionPane.showMessageDialog(this, error, "Validación",  
JOptionPane.WARNING_MESSAGE);  
        return;  
    }  
  
    String rol = rolSeleccionado.equalsIgnoreCase("Administrador") ? "ADMIN" : "CLIENTE";  
  
    Usuario u = new Usuario();  
    u.setNombre(nombre);  
    u.setUsername(usuario);  
    u.setPassword(password);
```

```
u.setRol(rol);
```

```
UsuarioControlador control = new UsuarioControlador();
```

```
if (control.registrarUsuario(u)) {
```

```
    JOptionPane.showMessageDialog(this, "Usuario registrado correctamente");
```

```
    if (modoRegistro) {
```

```
        SwingUtilities.getWindowAncestor(this).dispose();
```

```
    } else {
```

```
        limpiarFormulario();
```

```
    }
```

```
} else {
```

```
    JOptionPane.showMessageDialog(this, "El nombre de usuario ya existe");
```

```
}
```

```
}
```

```
private void btnLimpiarActionPerformed(java.awt.event.ActionEvent evt) {
```

```
    limpiarFormulario();
```

```
}
```

```
private void limpiarFormulario() {
```

```
    txtNombre.setText("");
```

```
    txtUsuario.setText("");
```

```
    txtPassword.setText("");
```

```
        cboRol.setSelectedIndex(0);  
    }  
}
```

```
// VARIABLES
```

```
private javax.swing.JButton btnGuardar;  
private javax.swing.JButton btnLimpiar;  
private javax.swing.JComboBox<String> cboRol;  
private javax.swing.JPanel jPanel1;  
private javax.swing.JLabel lblContraseña;  
private javax.swing.JLabel lblCorreo;  
private javax.swing.JLabel lblNombre;  
private javax.swing.JLabel lblRol;  
private javax.swing.JLabel lblTitulo;  
private javax.swing.JTextField txtNombre;  
private javax.swing.JPasswordField txtPassword;  
private javax.swing.JTextField txtUsuario;  
}
```

FrmLogin

```
package vista;
```

```
import vista.FrmPrincipal;  
import controlador.SistemaFacade;  
import java.awt.BorderLayout;  
import java.awt.Dimension;  
import java.awt.FlowLayout;  
import java.awt.GridBagConstraints;
```

```
import java.awt.GridBagLayout;

import java.awt.Insets;

import javax.swing.Box;

import javax.swing.JDialog;

import javax.swing.JOptionPane;

import javax.swing.JPanel;

import javax.swing.border.EmptyBorder;

import modelo.ControlSesion;

import modelo.Usuario;


// Patrón MVC: vista de inicio de sesión (punto de entrada de la aplicación).

// SOLID (S): responsabilidad única — solo gestiona la interfaz de login.

// SOLID (D): inversión de dependencias — depende de SistemaFacade (abstracción), no de
DAOs concretos.

public class FrmLogin extends javax.swing.JFrame {

    public FrmLogin() {

        initComponents();

        setLocationRelativeTo(null);

        setResizable(false);

    }

    private void initComponents() {

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

        setTitle("Iniciar sesión — SmartRail");

        getContentPane().setBackground(TemaSistema.getFondo());

    }

}
```



```
lblTitulo = new javax.swing.JLabel("SmartRail");  
lblTitulo.setFont(lblTitulo.getFont().deriveFont(java.awt.Font.PLAIN, 20f));  
lblTitulo.setAlignmentX(java.awt.Component.CENTER_ALIGNMENT);
```

```
lblCorreo = new javax.swing.JLabel("Usuario");  
lblPassword = new javax.swing.JLabel("Contraseña");  
txtUsuario = new javax.swing.JTextField(20);  
txtPassword = new javax.swing.JPasswordField(20);  
txtUsuario.setPreferredSize(new Dimension(260, 36));  
txtPassword.setPreferredSize(new Dimension(260, 36));  
txtPassword.addActionListener(this::txtPasswordActionPerformed);
```

```
btnIngresar = new javax.swing.JButton("Ingresar");  
btnRegistrar = new javax.swing.JButton("Registrarse");  
btnIngresar.setPreferredSize(new Dimension(130, 40));  
btnRegistrar.setPreferredSize(new Dimension(130, 40));  
btnIngresar.addActionListener(this::btnIngresarActionPerformed);  
btnRegistrar.addActionListener(this::btnRegistrarActionPerformed);
```

```
JPanel formCard = new JPanel();  
formCard.setBackground(TemaSistema.getFondoCard());  
formCard.setBorder(new EmptyBorder(32, 40, 32, 40));  
formCard.setLayout(new GridBagLayout());  
GridBagConstraints c = new GridBagConstraints();  
c.insets = new Insets(6, 8, 6, 8);
```

```
c.anchor = GridBagConstraints.LINE_START;
```

```
c.fill = GridBagConstraints.HORIZONTAL;
```

```
c.gridx = 0;
```

```
c.gridy = 0;
```

```
c.gridwidth = 2;
```

```
c.anchor = GridBagConstraints.CENTER;
```

```
c.weightx = 0;
```

```
formCard.add(lblTitulo, c);
```

```
c.gridx = 0;
```

```
c.gridy = 1;
```

```
c.gridwidth = 1;
```

```
c.anchor = GridBagConstraints.LINE_START;
```

```
formCard.add(Box.createVerticalStrut(22), c);
```

```
c.gridx = 0;
```

```
c.gridy = 2;
```

```
c.weightx = 0;
```

```
formCard.add(lblCorreo, c);
```

```
c.gridx = 1;
```

```
c.weightx = 1;
```

```
formCard.add(txtUsuario, c);
```

```
c.gridx = 0;
```

```
c.gridy = 3;
```

```
c.weightx = 0;
```

```
formCard.add(lblPassword, c);
```

```
c.gridx = 1;
```

```

c.weightx = 1;
formCard.add(txtPassword, c);
c.gridx = 0;
c.gridy = 4;
c.gridwidth = 2;
c.weightx = 0;
c.insets = new Insets(22, 8, 0, 8);
JPanel botones = new JPanel(new FlowLayout(FlowLayout.CENTER, 12, 0));
botones.setOpaque(false);
botones.add(btnIngresar);
botones.add(btnRegistrar);
formCard.add(botones, c);
JPanel center = new JPanel(new BorderLayout());
center.setBackground(TemaSistema.getFondo());
center.setBorder(new EmptyBorder(24, 24, 24, 24));
center.add(formCard, BorderLayout.CENTER);
getContentPane().setLayout(new BorderLayout());
getContentPane().add(center, BorderLayout.CENTER);

pack();
}

private void btnIngresarActionPerformed(java.awt.event.ActionEvent evt) {
    String username = txtUsuario.getText().trim();
    String password = new String(txtPassword.getPassword());
    if (username.isEmpty() || password.isEmpty()) {
        JOptionPane.showMessageDialog(

```

```

        this,

        "Debe ingresar usuario y contraseña",

        "Validación",

        JOptionPane.WARNING_MESSAGE);

    return;
}

// Patrón Facade + SOLID (D): accede a la lógica de negocio mediante la fachada
SistemaFacade fachada = SistemaFacade.getInstancia();

Usuario usuario = fachada.login(username, password);

if (usuario != null) {

    // Patrón Singleton: inicia la sesión global

    ControlSesion.iniciarSesion(usuario);

    JOptionPane.showMessageDialog(

        this,

        "Bienvenido " + usuario.getNombre(),

        "Acceso concedido",

        JOptionPane.INFORMATION_MESSAGE);

    FrmPrincipal principal = new FrmPrincipal();

    principal.setVisible(true);

    this.dispose();

} else {

    JOptionPane.showMessageDialog(

        this,

        "Usuario o contraseña incorrectos",

        "Error de autenticación",

        JOptionPane.ERROR_MESSAGE);

```

```

    }
}

private void txtPasswordActionPerformed(java.awt.event.ActionEvent evt) {
    btnIngresarActionPerformed(evt);
}

private void btnRegistrarActionPerformed(java.awt.event.ActionEvent evt) {
    JDialog dialog = new JDialog(this, "Registro de usuario", true);
    dialog.getContentPane().setBackground(TemaSistema.getFondo());
    dialog.setContentPane(new PnlUsuario(true));
    dialog.pack();
    dialog.setLocationRelativeTo(this);
    dialog.setVisible(true);
}

public static void main(String[] args) {
    java.awt.EventQueue.invokeLater(() -> {
        TemaSistema.aplicar();
        new FrmLogin().setVisible(true);
    });
}

// Variables declaration
private javax.swing.JButton btnIngresar;
private javax.swing.JButton btnRegistrar;
private javax.swing.JLabel lblPassword;
private javax.swing.JLabel lblTitulo;
private javax.swing.JPasswordField txtPassword;
private javax.swing.JTextField txtUsuario;

```

```
}
```

Paquete Controlador

Sistema Facade

```
package controlador;
```

```
import modelo.Usuario;
```

```
// Patrón Facade: punto de acceso unificado que oculta la complejidad de los controladores.
```

```
// Patrón Singleton: una sola instancia accesible globalmente mediante getInstancia().
```

```
// SOLID (S): responsabilidad única — coordinar la delegación entre controladores.
```

```
// SOLID (D): inversión de dependencias — la vista depende solo de esta fachada, no de controladores concretos.
```

```
public class SistemaFacade {
```

```
    // Singleton: instancia única
```

```
    private static SistemaFacade instancia;
```

```
    private final UsuarioControlador usuarioControl;
```

```
    // Singleton: constructor privado impide instanciación externa
```

```
    private SistemaFacade() {
```

```
        this.usuarioControl = new UsuarioControlador();
```

```
    }
```

```
    // Singleton: punto de acceso global con inicialización perezosa
```

```

public static SistemaFacade getInstancia() {
    if (instancia == null) {
        instancia = new SistemaFacade();
    }
    return instancia;
}

// Facade: delega al UsuarioControlador
public Usuario login(String username, String password) {
    return usuarioControl.iniciarSesion(username, password);
}

// Facade: delega al UsuarioControlador
public boolean registrarUsuario(Usuario u) {
    return usuarioControl.registrarUsuario(u);
}
}

```

Usuario Controlador

```
package controlador;
```

```
import java.util.List;
```

```
import modelo.Usuario;
```

```
import modelo.UsuarioDAO;
```

```
// SOLID (S): responsabilidad única — solo gestiona la lógica de negocio de usuarios.
```

```
public class UsuarioControlador {
```

```
private final UsuarioDAO dao;

public UsuarioControlador() {
    this.dao = new UsuarioDAO();
}

public Usuario iniciarSesion(String username, String password) {
    if (username == null || username.trim().isEmpty()
        || password == null || password.trim().isEmpty()) {
        return null;
    }
    return dao.login(username.trim(), password);
}

public boolean registrarUsuario(Usuario u) {
    if (u == null)
        return false;

    if (u.getUsername() == null || u.getUsername().trim().isEmpty())
        return false;

    if (u.getPassword() == null || u.getPassword().trim().isEmpty())
        return false;

    if (u.getRol() == null || u.getRol().trim().isEmpty())
        return false;

    return dao.registrar(u);
}

public List<Usuario> listarUsuarios() {
    return dao.listarUsuarios();
}
}
```


7. Pruebas

7.1 Caja Negra

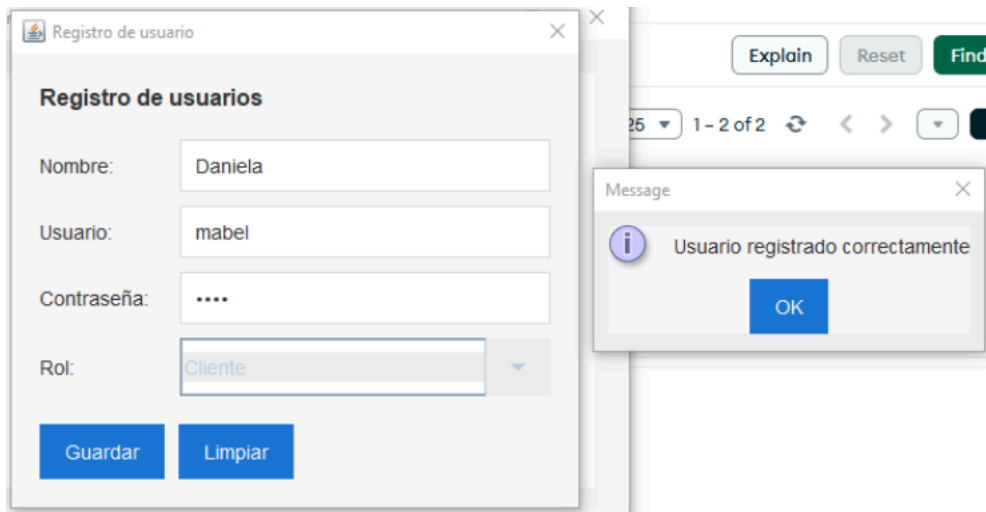
HISTORIAL DE USUARIO	
Nombre	REQ-001
Usuario	Administrador/Cliente
Nombre de la Historia	Autenticación de usuarios
Prioridad	Alta
Programador responsable	Daniela Narvaez
Descripción	El sistema permite registrar usuarios y posteriormente autenticar su acceso según su rol
Validación	El sistema registra usuarios en MongoDB y valida credenciales para permitir o restringir acceso

Registro de usuario

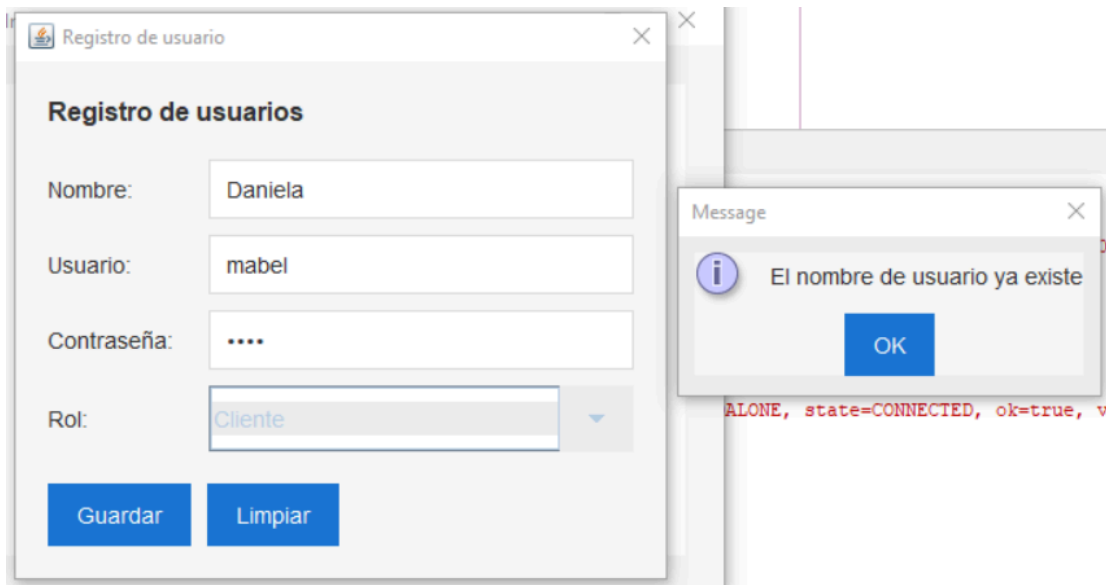
VARIABLE	CLASE DE EQUIVALENCIA	ESTADO	REPRESENTANTE
nombre	EC1: nombre no vacío	Válido	Daniela
nombre	EC2: nombre vacío	No válido	""
username	EC3: username único	Válido	mabel
username	EC4: username existente	No válido	mabel
password	EC5: contraseña no vacía	Válido	1234
password	EC6: contraseña vacía	No válido	""
rol	EC7: rol = CLIENTE	Válido	Cliente
rol	EC8: rol = ADMIN	Válido	Administrador

Registrar Usuario

PASS



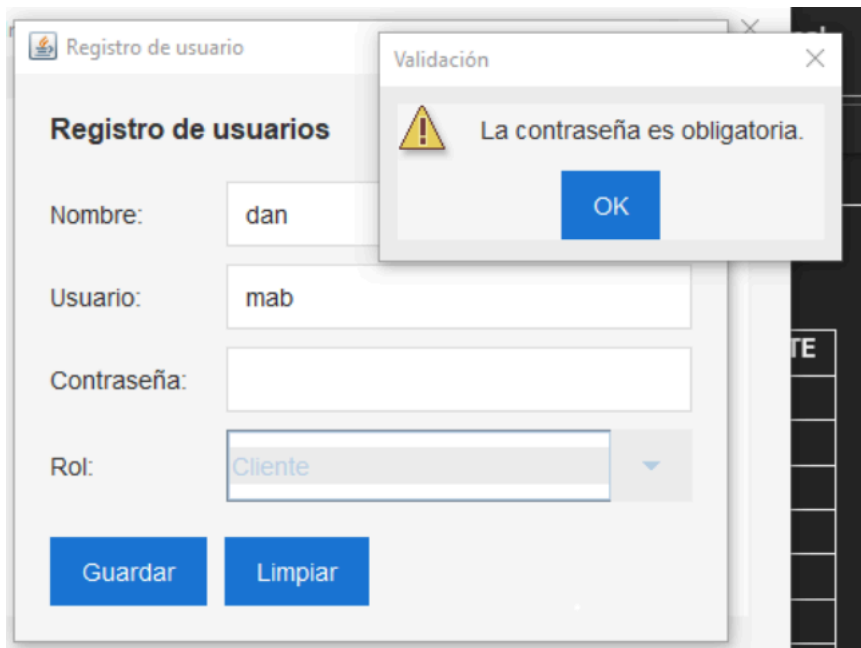
The screenshot shows a web application window titled "Registro de usuario". The form contains the following fields: "Nombre:" with the value "Daniela", "Usuario:" with the value "mabel", "Contraseña:" with masked characters "....", and "Rol:" with a dropdown menu showing "Cliente". At the bottom are two buttons: "Guardar" and "Limpiar". To the right, a "Message" dialog box displays the text "Usuario registrado correctamente" with an "OK" button. Above the message box, a navigation bar includes buttons for "Explain", "Reset", and "Find", along with a page indicator "1 - 2 of 2".



This screenshot shows the same "Registro de usuario" form as above, but with a different "Message" dialog box. The message reads "El nombre de usuario ya existe" (The username already exists). Below the dialog box, a line of red text is visible: "ALONE, state=CONNECTED, ok=true, v". The form fields and buttons remain the same.

Registrar Cliente

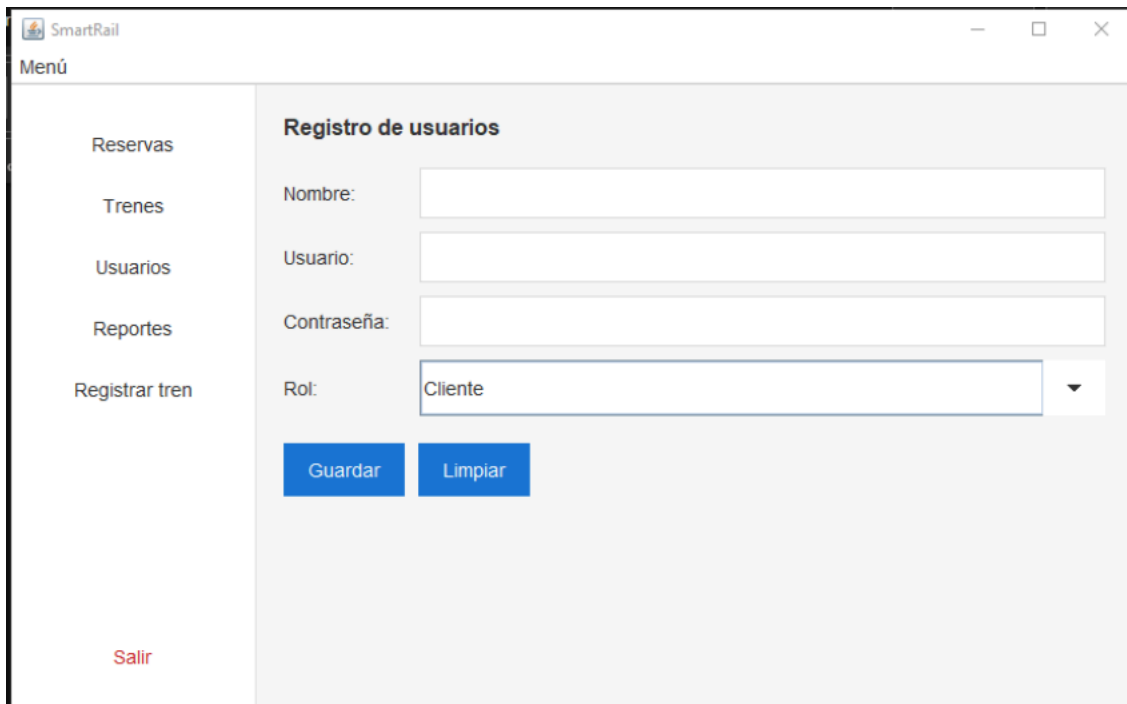
NO PASS



The image shows a 'Registro de usuario' (User Registration) dialog box. It contains the following fields: 'Nombre:' with the value 'dan', 'Usuario:' with the value 'mab', 'Contraseña:' (empty), and 'Rol:' with a dropdown menu showing 'Cliente'. At the bottom are 'Guardar' and 'Limpiar' buttons. A 'Validación' (Validation) error dialog is overlaid on top, displaying a yellow warning icon and the message 'La contraseña es obligatoria.' (Password is mandatory), with an 'OK' button.

Rol Administrador

PASS



The image shows the 'SmartRail' application interface. On the left is a 'Menú' (Menu) with options: 'Reservas', 'Trenes', 'Usuarios', 'Reportes', 'Registrar tren', and a red 'Salir' (Exit) button at the bottom. The main area is titled 'Registro de usuarios' and contains the same registration form as the previous image: 'Nombre:', 'Usuario:', 'Contraseña:', 'Rol:' (dropdown with 'Cliente'), and 'Guardar'/'Limpiar' buttons.

Rol Cliente

PASS

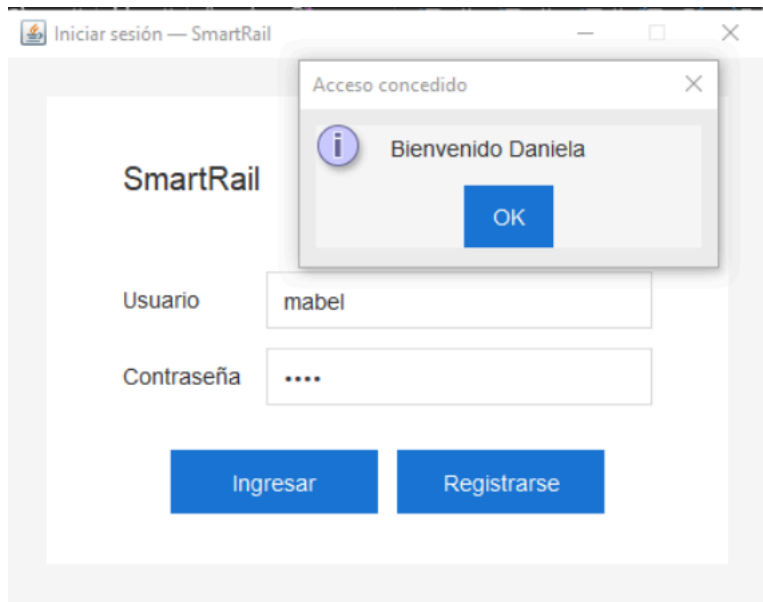


Inicio de Sesión

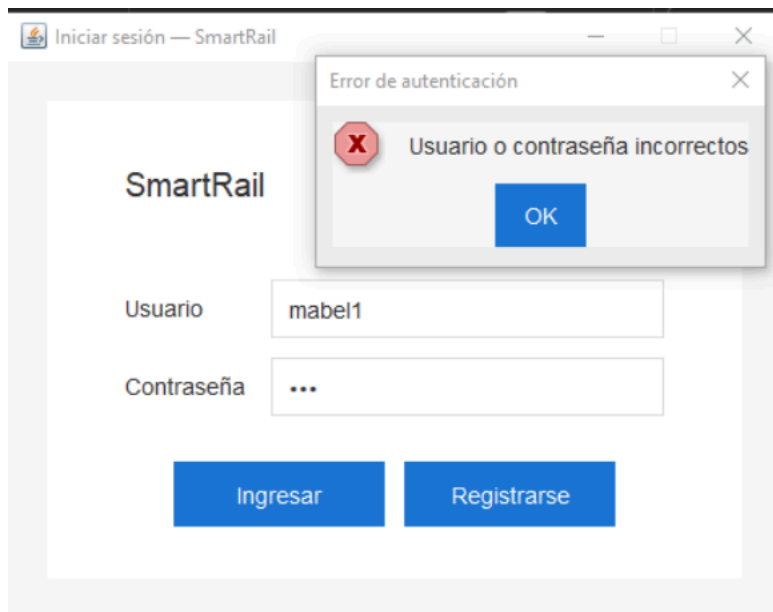
VARIABLE	CLASE DE EQUIVALENCIA	ESTADO	REPRESENTANTE
username	EC9: username correcto	Válido	mabel
username	EC10: username incorrecto	No válido	mabel1
password	EC11: contraseña correcta	Válido	1234
password	EC12: contraseña incorrecta	No válido	4321

Inicio de sesión usuario

PASS



NO PASS



8. Relación con CMMI

- ¿Qué prácticas de organización están aplicando?

En el sistema aplicamos prácticas formales de gestión y aseguramiento de calidad propias del

producto. Las principales prácticas son:

- Definición de roles y responsabilidades: El equipo está estructurado bajo la metodología ágil scrum, con roles claros como Product Owner, Scrum Master y Scrum Team.
- Gestión de requerimientos: Levantamiento y catalogación de requerimientos funcionales y no funcionales.
- Modelado del sistema: Identificar los actores y sus casos de uso para planificar interacciones del sistema.

• ¿Cómo gestionan el trabajo?

El equipo se organiza con ciclos de trabajo cortos llamados sprints, utilizando un cronograma de trabajo estructurado. Las funcionalidades se traducen a Historias de Usuario, las cuales se administran y priorizan en un Backlog.

Existen ceremonias y reuniones de trabajo que permiten controlar y encimar los avances del sistema.

• ¿Qué evidencias manejan?

- ERS
- Diagrama y descripciones de los flujos básicos de casos de uso
- Artefactos de gestión ágil
- Tablas de resultados y reportes de las pruebas del producto.
- Evidencias de levantamiento de información.

• ¿En qué nivel de madurez consideran que se encuentra el proyecto?

Nuestro proyecto se encuentra trabajando principalmente en el Nivel 2 y presenta características del Nivel 3 del modelo CMMI.

De acuerdo con el Nivel 2, el proyecto ya cuenta con actividades de planificación y control, lo que se evidencia en la asignación de roles, la definición del alcance y la identificación clara de los requerimientos antes de iniciar la fase de programación. Esto permite dar seguimiento al avance del proyecto y reducir la improvisación.

Asimismo, se observan elementos propios del Nivel 3, ya que algunos procesos se encuentran documentados y estandarizados, permitiendo que el equipo trabaje bajo una forma común de desarrollo y mantenga consistencia en la ejecución de las actividades.

9. Conclusiones

-Durante el desarrollo del sistema SmartRail, el equipo enfrentó algunas dificultades como la gestión del tiempo ya que coordinar las reuniones y sincronizar el avance del desarrollo iterativo entre todos los integrantes implicó un esfuerzo extra de planificación para cumplir con los sprints.

- La asignación de roles y responsabilidades facilitó la toma de decisiones.
- El levantamiento formal de requerimientos y la elaboración del diagrama de casos de uso nos sirvieron como guía para que el producto coincidiera con las necesidades del cliente
- Algo que se debe mejorar es la estandarización que se realiza en el nivel 3 de CMMI ya que es algo muy importante para el desarrollo correcto y completo del proyecto, para obtener buenos resultados.

10. Anexos

Características de los usuarios

Tipo de usuario	Usuario
Formación	NA
Actividades	Controlar y manejar el software(jugar)

Restricciones

- Interfaz para ser usada con internet.
- Uso de Dominio (X)
- Lenguajes y tecnologías en uso: JAVA.
- El sistema deberá tener un diseño e implementación sencilla, independiente de la plataforma o del lenguaje de programación.

Suposiciones y dependencias

- Se asume que los requisitos aquí descritos son estables
- Los equipos en los que se vaya a ejecutar el sistema deben cumplir los requisitos antes indicados para garantizar una ejecución correcta de la misma
- El equipo debe estar conectado al internet.
- Los periféricos de entrada deben funcionar al 100%.

Requisitos específicos

Requerimientos Funcionales

Identificación del requerimiento:	RF01
Nombre del Requerimiento:	Autenticación de usuario
Características:	Permitir que un cliente o administrador cree una cuenta en el sistema o acceso al sistema mediante credenciales válidas
Descripción del requerimiento:	Cliente y Administrador: Podrá registrarse mediante el ingreso de datos personales (nombre, correo y contraseña) y podrá iniciar sesión para ingresar al sistema, mediante el ingreso de usuario y contraseña.
Requerimiento NO funcional:	RNF01: Seguridad de autenticación RNF02: Interfaz intuitiva
Prioridad del requerimiento:	Alta

Identificación del requerimiento:	RF02
Nombre del Requerimiento:	Administración de clientes
Características:	Eliminar, editar clientes

Descripción del requerimiento:	Administrador: Podrá gestionar todos los usuarios que ingresen al sistema
Requerimiento NO funcional:	RNF03: Control de acceso por roles
Prioridad del requerimiento: Media	

Identificación del requerimiento:	RF03
Nombre del Requerimiento:	Gestionar Trenes
Características:	Crear, modificar o eliminar rutas (ya creadas) de los trenes.
Descripción del requerimiento:	Administrador: Podrá administrar la información de los trenes.
Requerimiento NO funcional:	RNF04: Validación de datos únicos (código de cada tren)
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF04
Nombre del Requerimiento:	Gestionar Horario
Características:	Crear, editar y eliminar horarios de trenes
Descripción del requerimiento:	Administrador: Podrá asignar y ver los horarios de salida y llegada cada tren del sistema
Requerimiento NO funcional:	RNF05: Validación de conflictos de horarios.
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF05
Nombre del Requerimiento:	Gestionar Reservas
Características:	Permite hacer reservas de boletos, consultar su estado (Reservado y Cancelado) y cancelarlas si aplica.
Descripción del requerimiento:	Cliente: Tendrá la posibilidad de reservar boletos de tren, consultar el estado de sus reservas y cancelarlas dentro de los plazos permitidos.
Requerimiento NO funcional:	RNF06: Alta Disponibilidad del sistema RNF07: Tiempo de respuesta rápida
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF06
--	------